Examen práctico
Programación Web Básica con React

Consumo de API simulada · TanStack Router · Git · Netlify

Modalidad	Individual
Duración sugerida	2 a 3 horas
Entrega final	Repositorio Git, URL de Netlify, commits progresivos y README breve
Temáticas posibles	Catálogo de videojuegos, biblioteca de cursos o dashboard de películas

Objetivo

Desarrollar una aplicación front-end en React que consuma datos sobre repuestos de vehículos, desde JSONBin, implemente navegación entre pantallas con TanStack Router, utilice componentes reutilizables y deje evidencia del proceso en Git y despliegue en Netlify.

1. Instrucciones generales para el estudiante

- Utilice su propio repositorio.
- Complete el examen de forma individual y realice commits conforme avance en cada paso del reto. Al final de todo enviar un Pull Request desde su rama de trabajo, a la rama main y dejarlo abierto.
- El proyecto debe funcionar localmente y en producción al momento de la entrega.
- La aplicación debe consumir datos desde el bin asignado y mostrarlos dinamicamente.
  En palabras simples, es un fetch en react. Lo complejo está en el realismo de la prueba, simularas el flujo de trabajo en un ambiente laboral.
- La URL del sitio desplegado y la del repositorio deben entregarse dentro del tiempo definido. (3 horas)

2. Guion resumido del ejercicio

1. Crear el repositorio base y el Projecto en VITE base. Hacer commit inicial.
2. Instalar dependencias y ejecutar el proyecto localmente.
3. Completar componentes base y estructura visual mínima.
4. Configurar las rutas principales con TanStack Router.
5. Conectar la app con el bin asignado en JSONBin.
6. Renderizar la lista de repuestos de forma dinámica a partir de la API (GET).
7. Agregar búsqueda con useState.
8. Manejar estados de carga (Loading), error.
9. Configurar variables de entorno y desplegar en Netlify.
10. Entregar el repositorio, la URL del sitio y el README final.

3. Temáticas del examen

Catálogo de repuestos de vehículos.

4. Detalles por cada parte

#	Reto	Descripción	Commit sugerido
1	Preparación **5 puntos**	Crear un repositorio público llamado examen1-progra4, crear un sitio web usando VITE, instalar dependencias y ejecutar localmente.	Hacer un commit inicial al repositorio remote, directamente a la rama principal. Mensaje: **“setup Project”**
2	Componentes base **10 puntos**	Completar Navbar, Footer, Home y CarParts.	Crear una rama **feature**/carparts-list Debe incluir los cambios relacionados a los nuevos componentes requeridos. Mensaje: **”create base components”**
3	Routing **20 puntos**	Configurar las rutas principales Home, CarParts y la navegación entre vistas.	Commit a la feature branch con el mensaje **“configure app routes”**
4	Consumo de API **20 puntos**	Realizar fetch al bin asignado y mostrar el listado principal. https://api.jsonbin.io/v3/b/69e535e236566621a8ce210a X-Access-Key $2a$10$7L0fDBh3v77EF1usWl4EfOwXzcST0EFg9vISOOTUPBq7xcutgDBU2 GET es un método HTTP de lectura. Asegurate de mostrar todos los elementos “Repuestos de Carro” de la lista. Pero si lo elementos de la lista superan 10, entonces debe salir un botón al final de la lista que diga ver más, y permita cargar los próximos 10 elementos de la lista.	mensaje : **"fetch and render data”**
5	Búsqueda **20 puntos**	Agregar buscador por nombre del repuesto.	mensaje: **add search field**
6	Estados de UI **15 puntos**	Manejar loading, error y empty state.	Mensaje: “handling error and corner cases”
7	Entorno y deploy **10 puntos**	Usar variables de entorno y desplegar en Netlify.	configura environment variables y completa el deployment

5. Configuración sugerida con JSONBin

Nota importante: la respuesta del fetch (HTTP GET) incluye los datos del bin dentro de response.record. Si el contenido guardado es {"items": [...]}, normalmente se accederá a response.record.items. Esto lo puedes confirmar tu mismo usando postman.

Agrega aquí alguna imagen de que lograste confirmar el tipo de objeto que retorna el API de JSONbin

Variables de entorno sugeridas:

```
CARPARTS_API_URL=https://api.jsonbin.io/v3/b/69e535e236566621a8ce210a
VITE_JSONBIN_ACCESS_KEY=$2a$10$7L0fDBh3v77EF1usWl4EfOwXzcST0EFg9vISOOTUPBq7xcutgDBU2
```

Ejemplo de lectura:

```
const response = await fetch(import.meta.env.VITE_API_URL)
const data = await response.json()
const items = data.record.items
```

6. Entregables

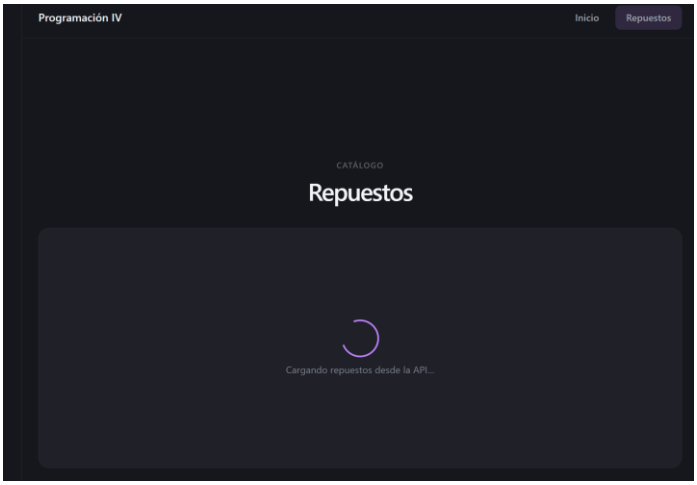
- Enlace al repositorio Git del examen.
- Enlace al sitio desplegado en Netlify. (Puede Agregar esos 2 enlaces a un .txt y subirlo a Profelink en el espacio de Examen).
- README dentro del repositorio re escrito con los cambios logrados y en donde se ubican para facilitar su revisión.
- Enviar un Pull Request (PR) a main de su propio repositorio, desde **feature**/carparts-list a main, dejar el PR abierto para poder hacer revisión.

7. Consideraciones del professor para determinar la madurez del entregado.

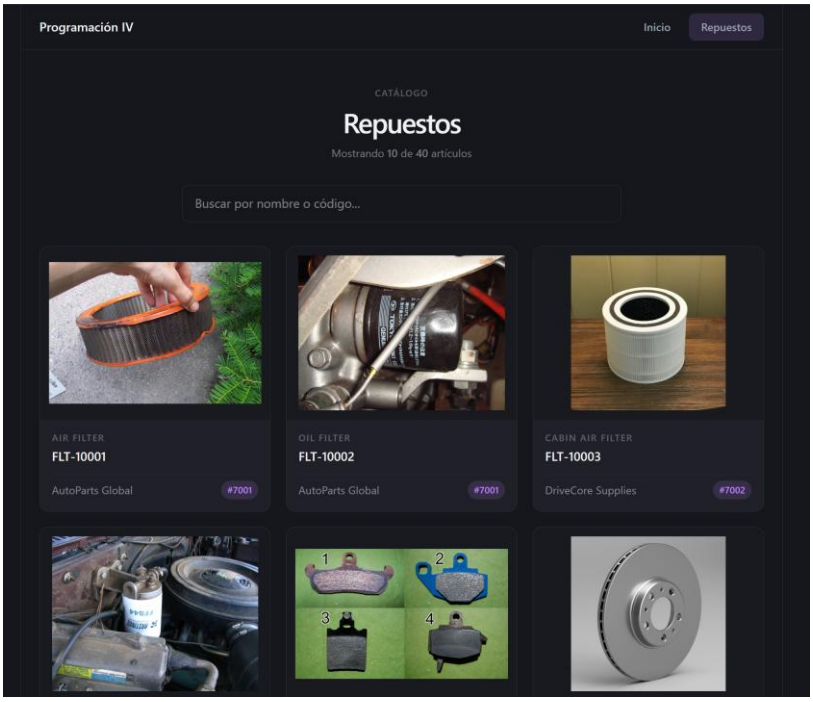
Criterio	Expectativa
Funcionalidad general	Que la app cumpla el flujo solicitado y no falle.
React y componentes	Buena separación, y reutilización.
Routing	Rutas funcionales y navegación correcta.
Fetch y manejo de datos	Consumo real del bin privado y render dinámico de la lista.
Búsqueda, Loading	Interacción útil con useState.
Git y calidad de commits	Evidencia de avance real y mensajes claros.
Deploy y Documentación	Sitio publicado y evidencia documentada de las pruebas de postman.

Prototipado Ejemplo:

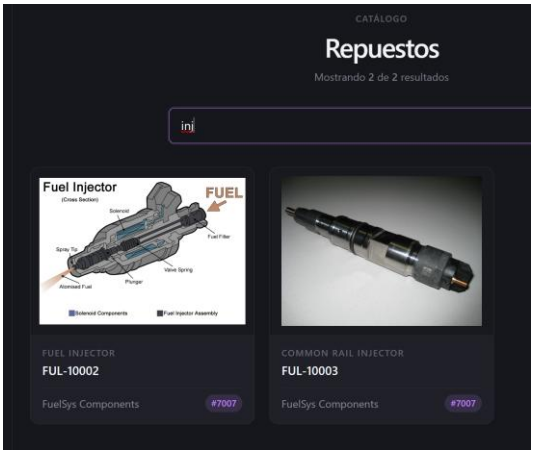
Loading



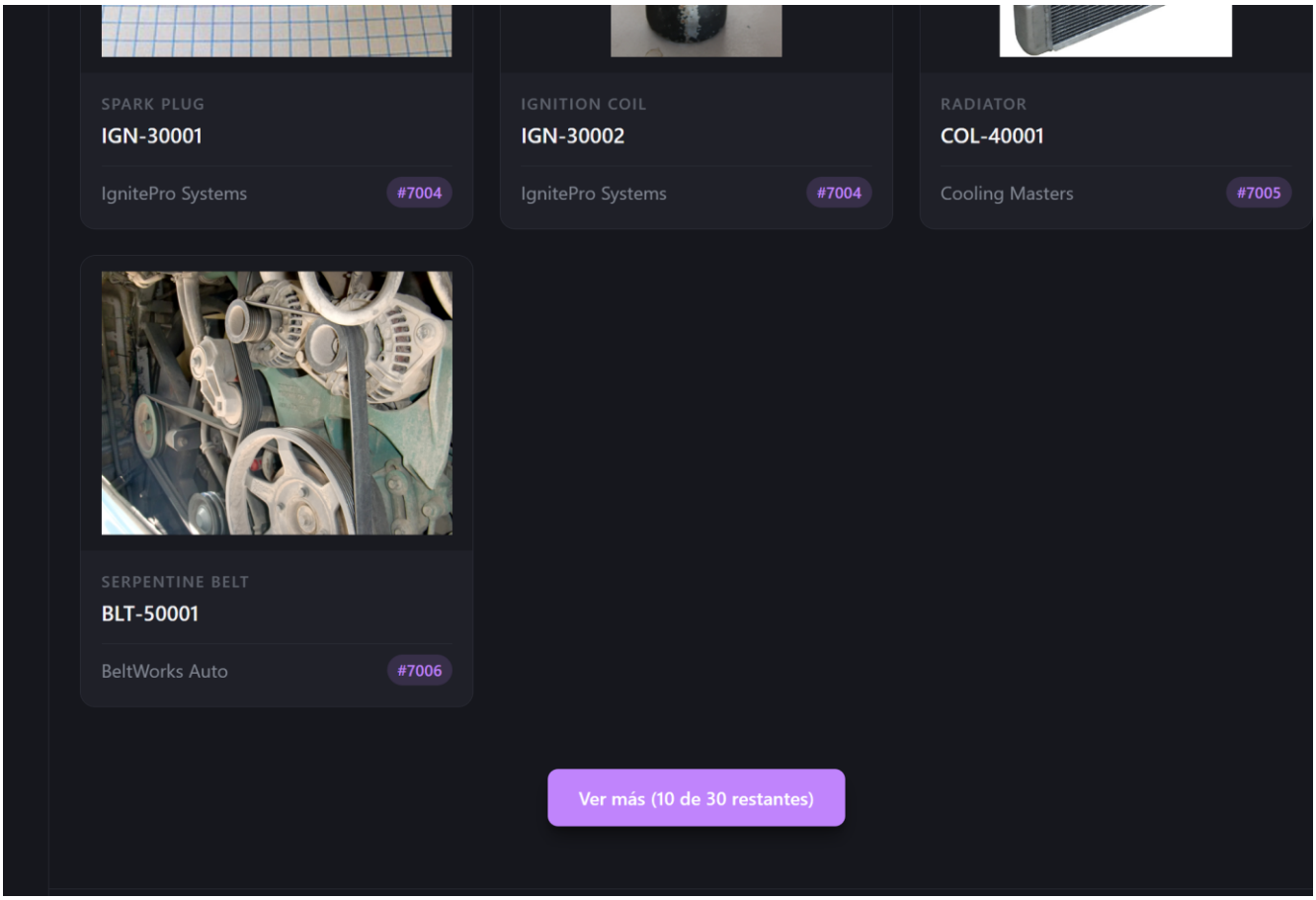
Carga Repuestos del API



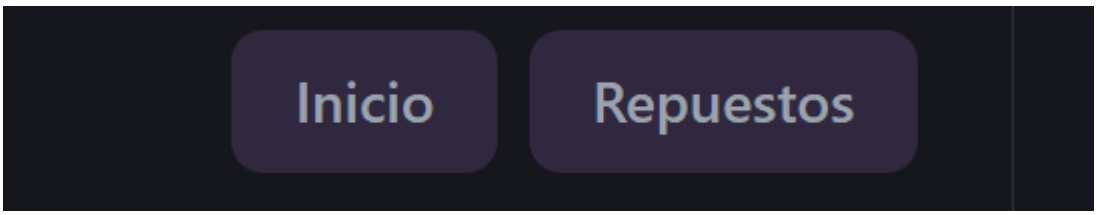
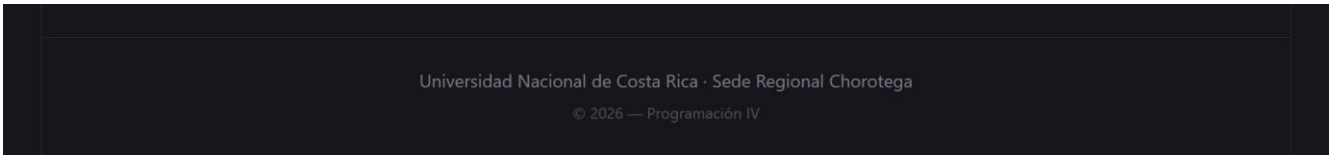
Busqueda Simple



Botón de Ver más:



Componente NavBar + Routing



Componente Footer: